

SourceBridge — Architecture

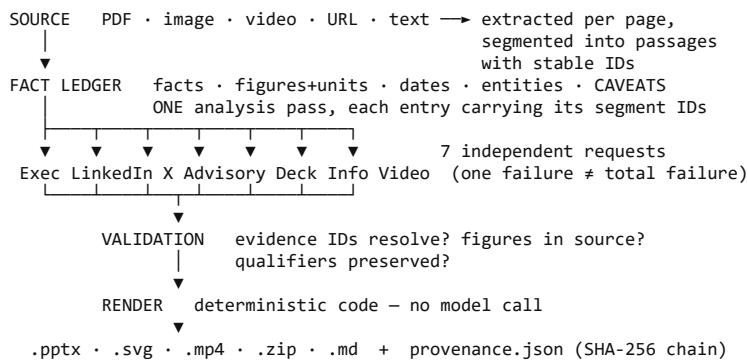
SIH 2026 · Problem statement 26154 · Theme: Blockchain & Cybersecurity

One source document becomes seven audience-specific communication artefacts sharing a single factual foundation, with every claim traceable to the passage it came from.

1. The problem

One incident report must become an executive summary, a public advisory, a briefing deck, a social post and a video script. Written separately they drift: a figure is rounded, a caveat dropped, "*attribution remains unconfirmed*" becomes "*attribution confirmed*". The number is rarely wrong — the **certainty around it** is. Asking a chatbot five times reproduces that drift, because each answer is independent. The architecture, not the model, is what prevents it.

2. Pipeline



3. Three guarantees

Consistency. Extraction and fact analysis run *once*; every format reads the same ledger. Independent generation is what causes drift, so the architecture removes the independence.

Exactness. The model returns structured JSON against a Zod schema. Slides, SVG and every video frame are drawn by application code — a model is never asked to render a figure, so it cannot render one wrongly. Exports make no model call, so an export cannot fail because a provider is down.

Meaning. A numeric check catches a figure that changed; it is blind to a figure that stayed while the certainty around it did not. `lib/meaningDrift.ts` pairs each cited output block with its source passage and reports a **dropped qualifier** as a warning (source says "*preliminary*", output does not) and an **escalated claim** as an error ("*some*" → "*all*", "*unconfirmed*" → "*confirmed*"). Deterministic, negation-aware string analysis — no model is asked whether the meaning changed, because that answer could not be verified.

4. Stack

Layer	Choice	Rationale
Application	Next.js 16 · React 19 · TypeScript	One deployable unit; credentials stay server-side
Schemas	Zod 4	One definition drives structured output <i>and</i> validation
PDF text	unpdf	Per-page text, which the evidence model depends on
Deck / infographic	PptxGenJS · hand-written SVG	Native editable charts; deterministic output
Video	@resvg/resvg-js + ffmpeg	Frames drawn locally; provider TTS gives narration only
Providers	Groq, then Google Gemini	Separate allowances; order chosen by measurement

No database, queue or vector store — none is justified at this scope.

5. Evidence and provenance

Every passage carries a stable ID (`src-<hash>-p<page>-<n>`). Generated content cites those IDs, the server resolves them and **discards any the model invented**, and clicking a claim opens the exact passage behind it.

Each export ships `provenance.json`: a SHA-256 chain over the source text, the fact ledger and every artefact, each entry sealed over the one before it. Altering any recorded content breaks verification at that entry and identifies which.

This is a **hash chain, not a blockchain**. It establishes integrity and ordering. It does not prove the source document was authentic, and nothing is anchored to an external ledger — stated plainly in the file itself and in both READMEs.

6. Security and reliability

Credentials are read in **one file** (`lib/provider.ts`, `server-only`) and never reach the browser. Source content is **data, never instructions** — uploaded text is fenced and marked as data, so directives inside a document are treated as quoted content; covered by tests. URL fetching resolves and screens every host against private, loopback, link-local and CGNAT ranges **before each request and after every redirect**, reading bodies against a running byte cap. Uploads are size- and type-checked before being read.

Text generation walks one chain across two providers, ordered by **measurement, not tier**: every candidate was benchmarked against the real schemas for schema validity, figure preservation and qualifier retention, and two were removed on evidence — one reproduced half the source figures across repeated runs, another took 180 seconds and returned invalid JSON. Each call runs under a deadline, a refusing model enters a process-wide cooldown so seven formats do not each rediscover it, a stated rate-limit delay is honoured rather than guessed, and a request larger than a model's allowance rotates immediately rather than retrying what cannot succeed.

7. Scope — what this does *not* do

Stated because the interface is not permitted to imply otherwise.

- **No fact verification.** Validation is structural: IDs resolve, figures appear in the source, qualifiers survive, content fits its layout. It does not check whether content is *true*.
- **No OCR.** Scanned PDFs are refused with a clear message. Images are read by a vision model — a transcription, not an extraction — and labelled as such.
- **No generative imagery or video.** Frames are drawn by code: a generative model cannot be trusted with a figure, and in a video the viewer cannot check it.
- **No confidence scores, persistence, accounts or approval workflow.** Any confidence number shown would be invented; work lives in the browser tab.

8. Verification and deployment

247 automated tests run without an API key, covering extraction, schema validation, evidence resolution, meaning drift, the provenance chain, prompt-injection boundaries, SSRF screening and every renderer. Exports are checked further by inspecting the produced OOXML and SVG, and by probing rendered video for valid H.264/AAC streams. Deploys as a single Next.js container; `ffmpeg` on the host enables MP4 rendering, and without it the application says so and offers the video package instead.